



Light Brain

ESP32 & UNO Mecanum Wheel Smart Car

Table of contents

1. Product Introduction.....	3
1.1. describe.....	3
1.2. important.....	3
2. Experience product features.....	5
2.1. Description.....	5
2.2. Upload UNO code.....	5
2.3. Upload ESP32 code.....	6
2.4. Check IP address.....	9
2.5. Realize remote control function.....	10
3. Get to know the kit.....	12
3.1. UNO development board resource distribution.....	12
3.2. ESP32 Resource Distribution.....	12
3.3. Expansion board interface and resource distribution.....	12
3.4 Mecanum wheel.....	13
How does ultrasound work?.....	14
4. Code analysis.....	17
4.1. 1_UNO.ino receiving command control.....	17
4.2. WiFi Remote Control 2_ESP32.ino.....	26

1. Product Introduction

1.1. Describe

This product is based on ESP32 and UNO master control. It connects to the WiFi hotspot through ESP32, establishes a Web service page, and transmits the camera image to the web terminal. ESP32 communicates with UNO through the serial port, thereby controlling the movement of the smart car on the Web terminal.

1.2. Important

ESP32 has high requirements for voltage stability. During normal operation, its main power source is the VCC/GND port of the expansion board plugged into the UNO, and RX/TX is used as a data transmission port. Generally, the 7~9V power supply from the battery box is supplied to the UNO board through the DC port, then stabilized to about 5V by the expansion board above, and then supplied to ESP32 through VCC.

In special cases, if the ESP32 input does not reach about 5V, the WiFi connection will be disconnected. In this case, you should consider insufficient battery power or other line connection errors.

After downloading the tutorial file package, please unzip the file to your

desktop or a commonly used computer disk. Otherwise, if you put it in the default download location and do not unzip it, the Arduino IDE on some newer computers cannot correctly identify the default downloaded compressed file storage path location, resulting in code compilation failure!

2. Experience product features

2.1. Description

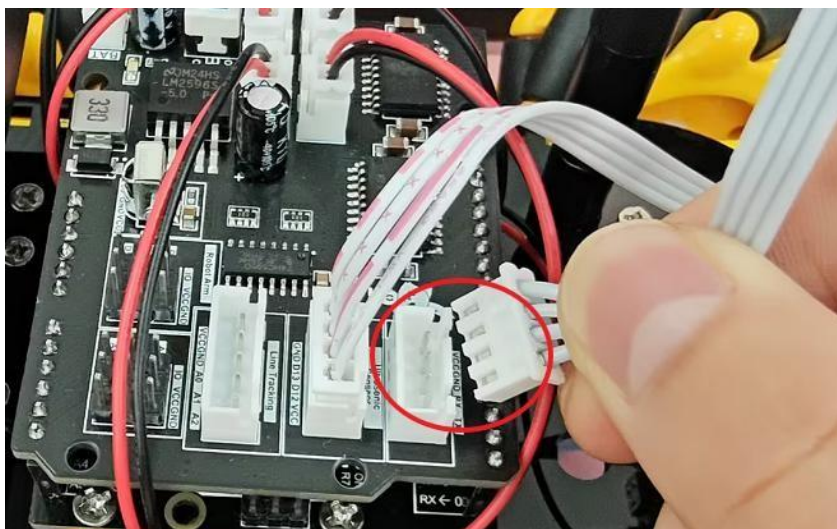
This section mainly introduces the use of UNO and ESP32 CAM, learns how to upload code and realize that after ESP32 CAM is connected to WiFi, it can display the captured images to the web side in real time and control the driving of the smart car on the web page .

2.2. Upload UNO code

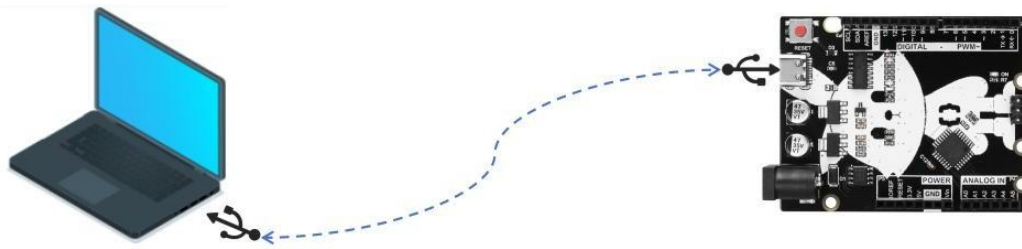
Open the code file (folder path: 2_Arduino_Code\1_UNO\1_UNO.ino)

arduino	2/7/2026 1:50 PM	File folder
esp32	2/7/2026 1:50 PM	File folder
README	2/7/2026 1:43 PM	Markdown file

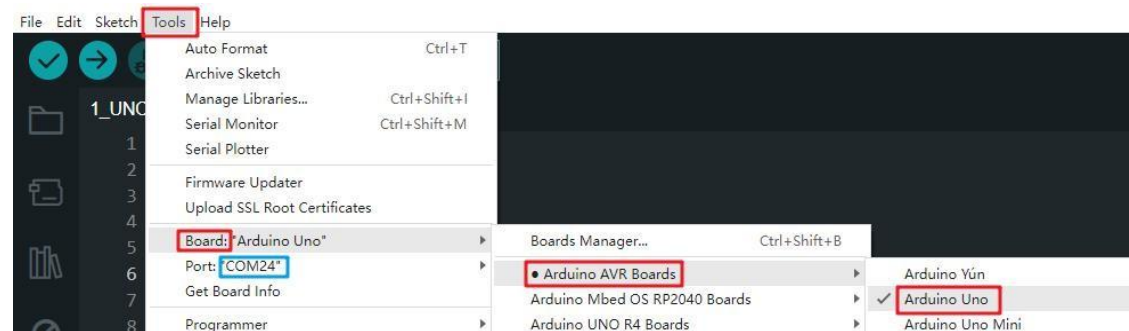
Unplug the RX/TX cables before uploading the program (as shown below) , and plug them back in after uploading. (**Be careful not to pull or plug the cables too much to prevent damaging the ESP32 board at the other end**)



Connect the uno board to the computer with a USB cable

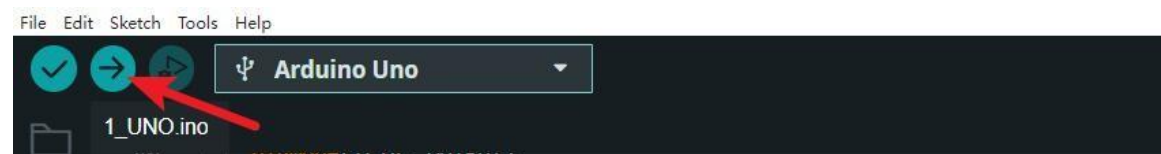


Select the board type as UNO and the serial port number as COM24

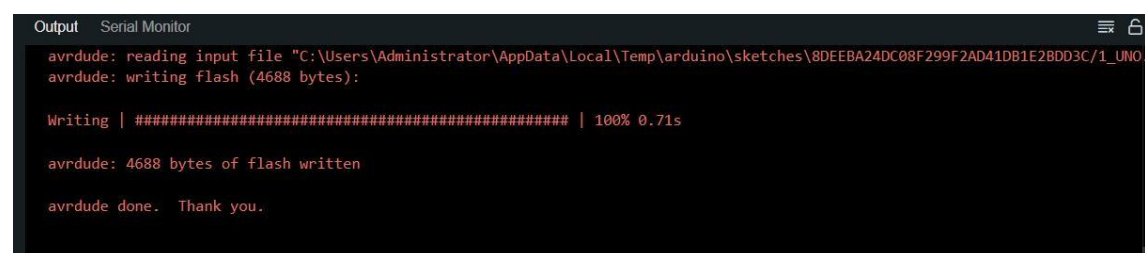


Note: The serial port displayed is different for everyone , although COM 24 is selected here , it may be COM3 or COM4 on your computer.

Click the "Upload" button and the program starts uploading.



After the upload is successful, the message “Done uploading” will be displayed.



2.3. Upload ESP32 code

Open the code file (path: 2_Arduino_Code\2_ESP32\2_ESP32.ino)



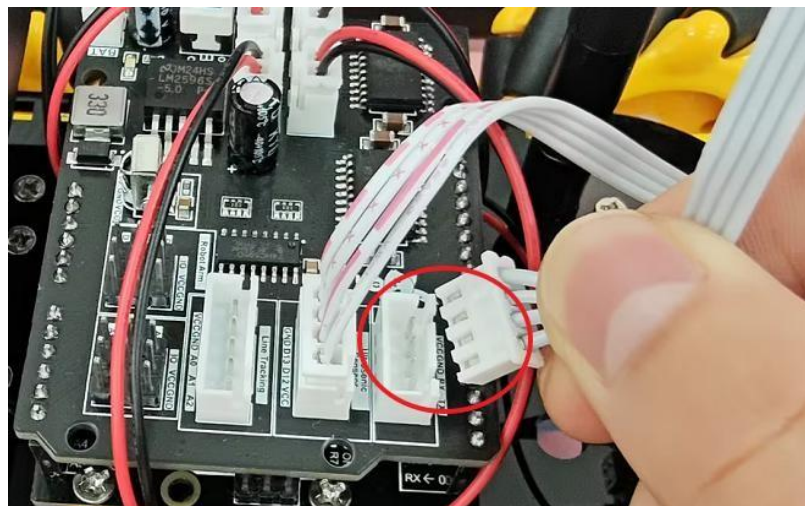
network credentials account and password variables in the code to your own WiFi name and password !

```
10 //Replace with your network credentials
11 const char* ssid = " ";
12 const char* password = " ";
```

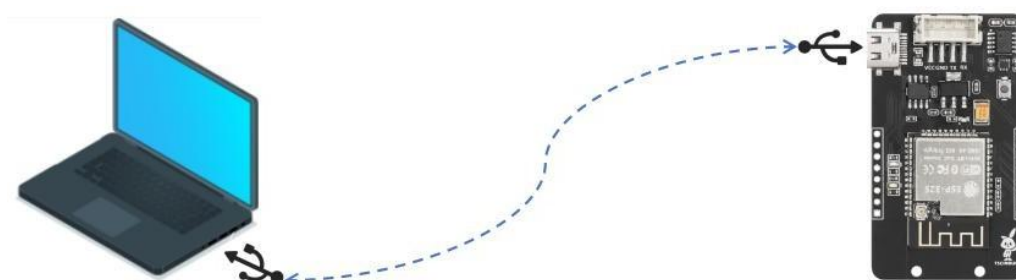
At the same time, you should check that the WiFi that ESP32 is connecting to is 2.4G instead of 5G band, otherwise the function cannot be realized!

ssid= "ZHIYI" ✓ ssid= "ZHIYI-5G" ✗

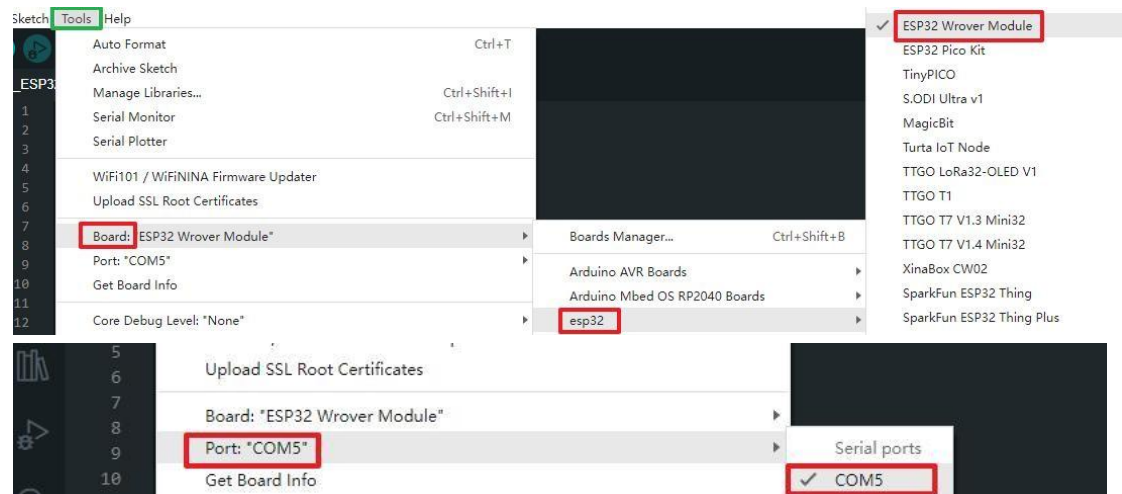
Similarly, when the ESP32 is powered externally, it will affect the USB port programming, so unplug the cable before uploading the program (as shown below) , and plug it back in after the upload is complete.



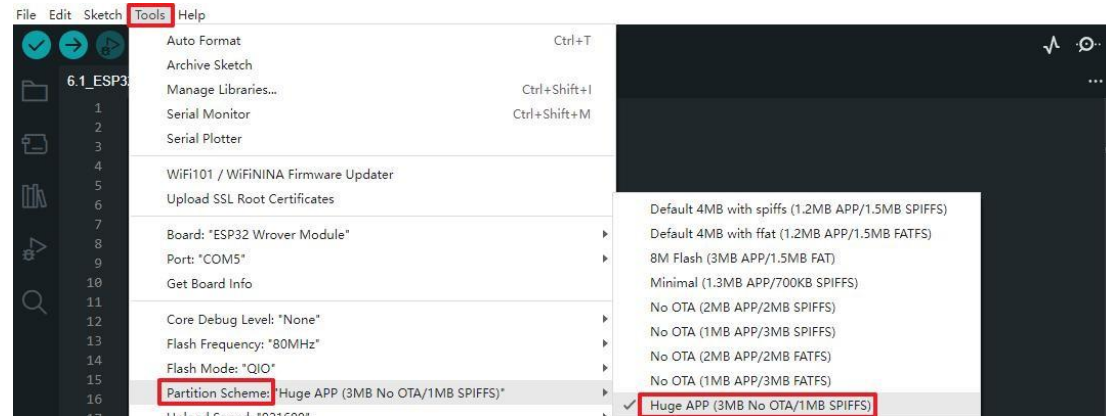
Connect the ESP32 board to your computer using a USB cable



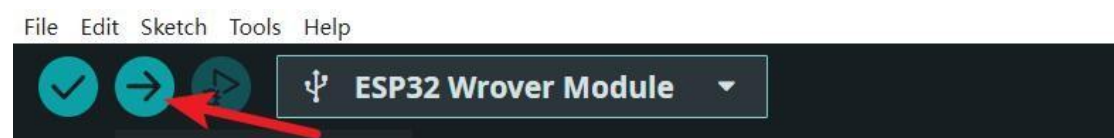
Select the board type as **ESP32 Wrover Module** and the serial port number as COM5



Note: Select **ESP32 Wrover** as the board type and **COM5** as the serial port . Actually, the serial port displayed by each person is different, it may be COM3 or COM4 on your computer.



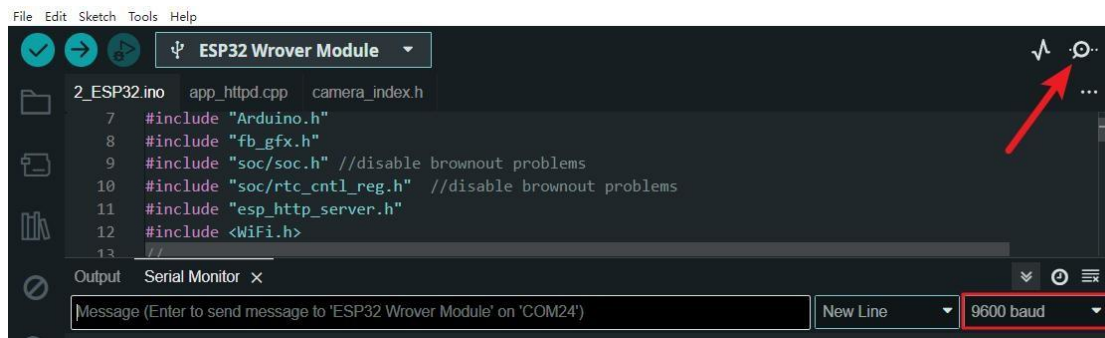
Click the "Upload" button and the program starts uploading.



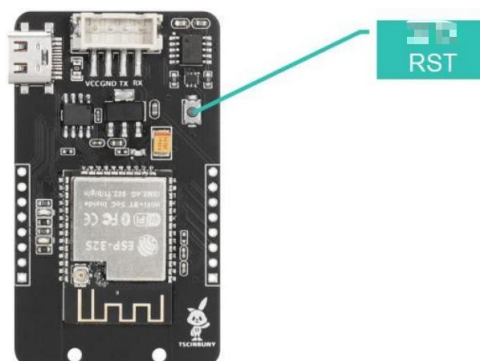
After the upload is successful, the message "Done uploading" will be displayed.

2.4. Check IP address

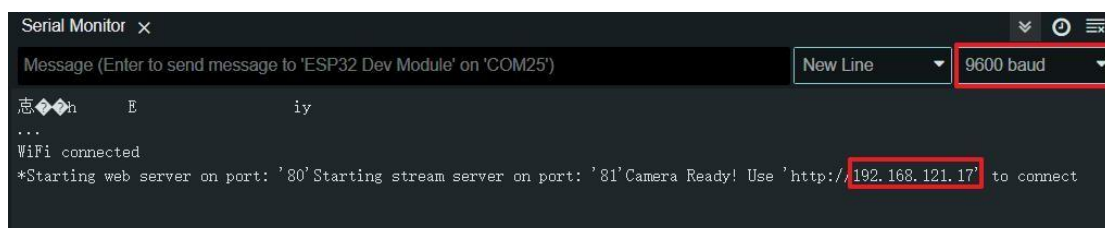
Open the serial monitor and set the baud rate to 9600 in the lower right corner



Press the reset button on the ESP32 CAM to reload the program. When it is just reset, the smart car will start moving by default and will not stop until it successfully connects to WiFi. This is normal.



When the program reloads and successfully connects to WiFi, the IP address will be displayed: <http://xxx.xxx.xxx.xxx> , please write it down .



Use your mobile phone to connect to the same WiFi (that is, the SSID filled in the code) and fill in the IP address into the device's browser to

access the service.

Note: Restart ESP32 after each power outage. If you cannot see the shooting picture on the browser, you need to press RST to reset it once to connect to WiFi faster.

2.5. Realize remote control function

Use your phone to connect to the same WiFi (that is, the ssid filled in the code) and fill in the IP address into the device's browser to access the service. While viewing the camera image, press the button to control the movement of the car. The bottom button controls the movement speed.

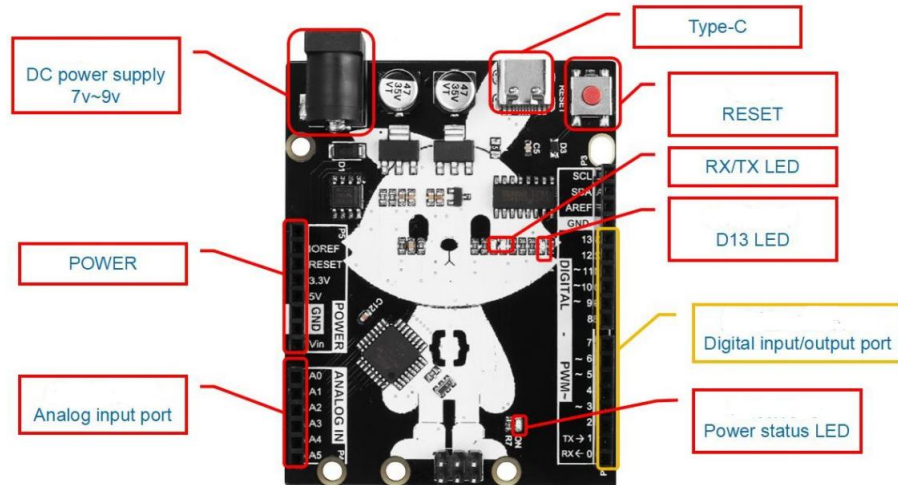
When you can't see the camera image:

1. Please check whether the WiFi name and password in the ESP32 code have been modified correctly;
2. Check whether the network signal is good and the power supply of the car is normal;

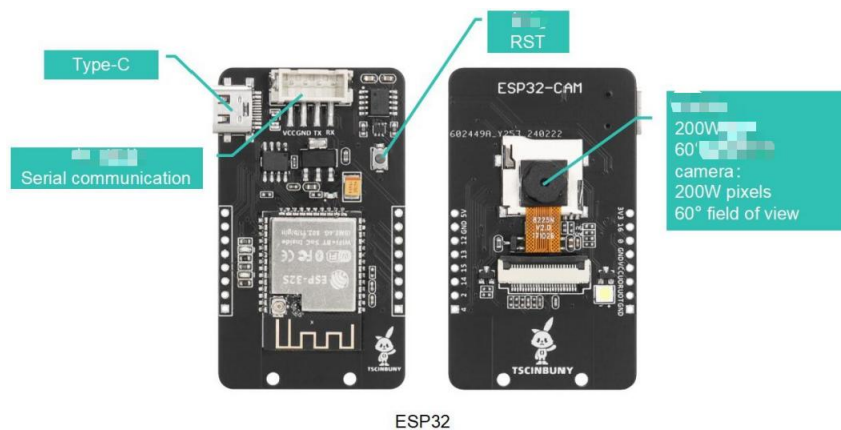
3. Avoid operating under the sun, as the high temperature from solar radiation will affect the stability of ESP32' s connection to WiFi.
4. Press the reset button on the ESP32 and wait for the WiFi connection.
(The mobile phone and ESP32 must be connected to the same WiFi router)

3. Get to know the kit

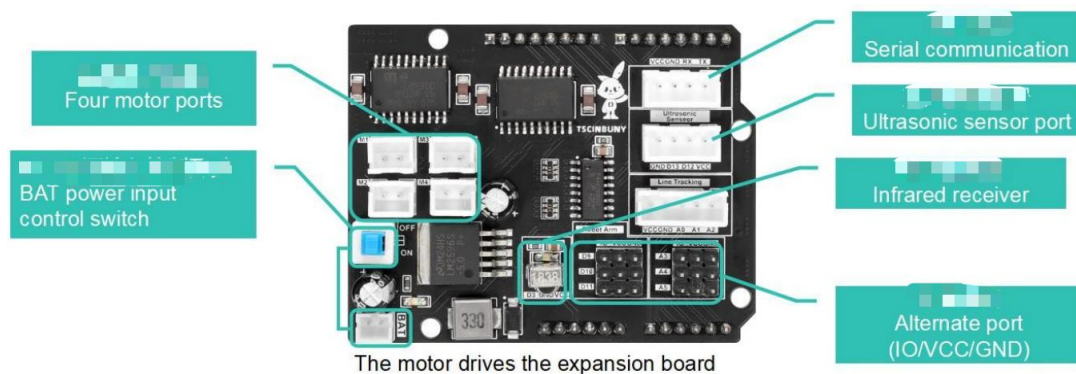
3.1. UNO development board resource distribution



3.2. ESP32 Resource Distribution



3.3. Expansion board interface and resource distribution



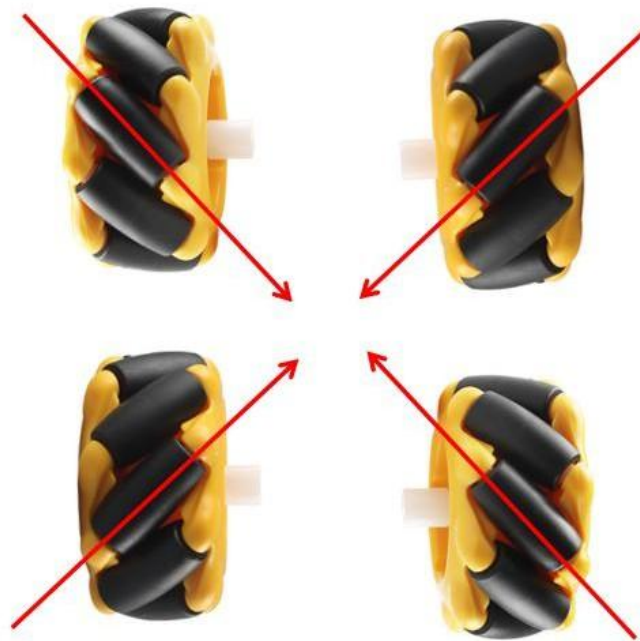
3.4 Mecanum wheel



A Mecanum wheel is a wheel with a peripheral axle. There are two types of wheels, one with a peripheral axle tilted to the left and the other with a peripheral axle tilted to the right. These angled peripheral axles convert part of the wheel steering force into a wheel normal force, so the car can achieve left and right translation.

Assembly points (top view):

The peripheral axle points to the center of the car



Ultrasound

Ultrasonic ranging is a very useful and widely used measurement method.

This lesson mainly introduces the working principle of ultrasonic module, masters the connection of ultrasonic circuit diagram and learns the method of ultrasonic module ranging through programming.

HC-SR04 Ultrasonic Module:



Working voltage	DC-5V
Working current	15mA
Working frequency	40KHz
Distance range	2cm ~ 400cm
Measuring angle	15 °
Input trigger signal	10 US TTL pulse
Output echo signal	Output TTL level signal, proportional to the range

How does ultrasound work?

1. The transmitter (trig pin) sends a signal: high-frequency sound;
2. When the signal hits an object, it is reflected;
3. Receiver (echo pin): Receives the signal reflected back from it.

There are many methods of ultrasonic ranging. The principle of this system in ultrasonic measurement is: detect the transmission time of ultrasonic waves from the ultrasonic transmitter through the gas medium to the receiver , multiply this time by the speed of sound in the air , and obtain the distance the sound propagates.

The ultrasonic transmitter emits ultrasonic waves in a certain direction, and the MCU starts timing at the same time. The ultrasonic waves are emitted in the air, and return immediately when encountering an obstacle on the way. The ultrasonic receiver stops timing immediately after receiving the reflected wave.

T recorded by the timer , the distance (s) from the launch point to the obstacle can be calculated .

$$\text{Formula: } S = VT/2$$

Four factors limit the maximum measurable distance of an ultrasonic system: the amplitude of the ultrasonic wave, the texture of the reflector, the angle between the reflected and incident sound waves, and the sensitivity of the receiving transducer. The ability of the receiving transducer to directly receive the sound pulse will determine the minimum measurable distance.

Ultrasonic detection distance code:

```
int SR04(int Trig ,int Echo)
```

```
{
```

```
float cm = 0;
```

```
digitalWrite(Trig,LOW);
```

```
delayMicroseconds(2);
```

```
digitalWrite(Trig,HIGH);
```

```
delayMicroseconds(15);
```

```
digitalWrite(Trig,LOW);
```

```
cm = pulseIn(Echo,HIGH)/58.8;
```

```
// Serial.print("Distance:");
```

```
//Serial.print(cm);
```

```
// Serial.println("cm");
```

```
delay(10);
```

```
return cm;
```

```
}
```


4. Code analysis

4.1. 1_UNO.ino receiving command control

```
//PWM control pin

#define PWM1_PIN 5

#define PWM2_PIN 6

// 74HCT595N chip

#define SHCP_PIN 2 // Clock shift pin

#define EN_PIN 7 // Enable pin

#define DATA_PIN 8 // Serial data

#define STCP_PIN 4 // Memory register clock

//Ultrasonic control pin

const int Trig = 12;

const int Echo =13;


//Define the car motion control value

const int Forward = 92; // forward

const int Backward = 163; // Backward

const int Left = 149; // Shift left

const int Right = 106; // right translation

const int Top_Left = 20; // Move to the upper left

const int Bottom_Left = 129; // Move to the bottom left
```

```
const int Top_Right = 72; // Move to the top right

const int Bottom_Right = 34; // Move to the bottom right

const int Turn_Left = 172; // Turn left

const int Turn_Right = 83; // Turn right

const int Stop = 0; // Stop


int distance = 0; //Variable to store the ultrasonic sensor measurement
value

char serialData; //Store the data received by the serial port in serialData

char cmd; //Store Bluetooth receiving command


void setup()

{

Serial.begin(9600); //Set the serial port baud rate to 9600


//Configure as output mode

pinMode(SHCP_PIN, OUTPUT);

pinMode(EN_PIN, OUTPUT);

pinMode(DATA_PIN, OUTPUT);

pinMode(STCP_PIN, OUTPUT);

pinMode(PWM1_PIN, OUTPUT);

pinMode(PWM2_PIN, OUTPUT);
```

```
pinMode(Trig, OUTPUT);
```

```
pinMode(Echo, INPUT);
```

```
pinMode(PWM1_PIN,OUTPUT);
```

```
pinMode(PWM2_PIN,OUTPUT);
```

```
pinMode(Trig,OUTPUT); //Set Trig to output
```

```
pinMode(Echo,INPUT); //Set Echo as input
```

```
Motor(Stop,0); //Initialize parking
```

```
}
```

```
void loop()
```

```
{
```

```
distance = SR04(Trig,Echo); //Ultrasonic distance acquisition
```

```
GetCommandEsp32(); //Call esp32 car control function
```

```
}
```

```
// Motor driving function;
```

```
void Motor(int Dir, int Speed1)
{
    digitalWrite(EN_PIN, LOW);
    analogWrite(PWM1_PIN, Speed1);
    analogWrite(PWM2_PIN, Speed1);

    digitalWrite(STCP_PIN, LOW);
    shiftOut(DATA_PIN, SHCP_PIN, MSBFIRST, Dir);
    digitalWrite(STCP_PIN, HIGH);
}

//Ultrasonic distance acquisition function
int SR04(int Trig ,int Echo)
{
    float cm = 0;

    digitalWrite(Trig,LOW);
    delayMicroseconds(2);
    digitalWrite(Trig,HIGH);
    delayMicroseconds(15);
    digitalWrite(Trig,LOW);

    cm = pulseIn(Echo,HIGH)/58.8;
```

```
// Serial.print("Distance:");

//Serial.print(cm);

// Serial.println("cm");

delay(10);

return cm;

}


//Serial port data receiving function

void GetCommandEsp32()

{

if(Serial.available() > 0) //Determine whether the received data is greater
than 0

{

serialData = Serial.read(); //Automatic receiving function

Serial.println(serialData);

if ('F' == serialData ) cmd = 'F'; //Save the characters received by the
serial port to CMD

else if('B' == serialData ) cmd = 'B';

else if('L' == serialData ) cmd = 'L';

else if('R' == serialData ) cmd = 'R';

else if('A' == serialData ) cmd = 'A';

else if('D' == serialData ) cmd = 'D';
```

```
else if('S' == serialData ) cmd = 'S';  
else if('Q' == serialData ) cmd = 'Q';  
else if('W' == serialData ) cmd = 'W';  
else if('Z' == serialData ) cmd = 'Z';  
else if('X' == serialData ) cmd = 'X';  
}
```

```
if('F' == cmd) //If the string F is received, the car moves forward and can  
avoid obstacles
```

```
{  
AvoidingObstacles(); //Call the function to realize the function of moving  
forward and automatically avoiding obstacles  
// Motor(Forward,250);  
}
```

```
else if('B' == cmd) //If string B is received, execute back
```

```
{  
Motor(Backward,250);  
}
```

```
else if('L' == cmd) //If the string L is received, perform left translation
```

```
{  
Motor(Left,250);  
}
```

```
else if('R' == cmd) //If the string R is received, perform right translation
```

```
{
```

```
Motor(Right,250);
```

```
}
```

```
else if('A' == cmd) //If string A is received, perform left rotation
```

```
{
```

```
Motor(Turn_Left,250);
```

```
}
```

```
else if('D' == cmd) //If string D is received, perform right rotation
```

```
{
```

```
Motor(Turn_Right,250);
```

```
}
```

```
else if('S' == serialData) //If string S is received, stop the car
```

```
{
```

```
Motor(Stop,0);
```

```
}
```

```
else if('Q' == serialData) //If the string Q is received, move to the upper
```

```
left
```

```
{
```

```
Motor(Top_Left,250);
```

```
}
```

```
else if('W' == serialData) //If the string W is received, move it to the
upper right
{
Motor(Top_Right,250);
}

else if('Z' == serialData) //If the string Z is received, move left and right
{
Motor(Bottom_Left,250);
}

else if('X' == serialData) //If string X is received, move right down
{
Motor(Bottom_Right,250);
}
}

//Obstacle avoidance function
void AvoidingObstacles()
{
if((distance > 20 ) || cmd == 'F') //If the distance is greater than 20cm or
the Bluetooth receives a command equal to F
{
delay(100);
```



```
if(distance > 20) // Determine if the distance is really greater than 20cm
{
    Motor(Forward,250); //Move forward
}

else //Otherwise, execute backward and rotate left
{
    Motor(Backward,250); //Execute backward

    delay(500);

    Motor(Turn_Left,250); //Left rotation

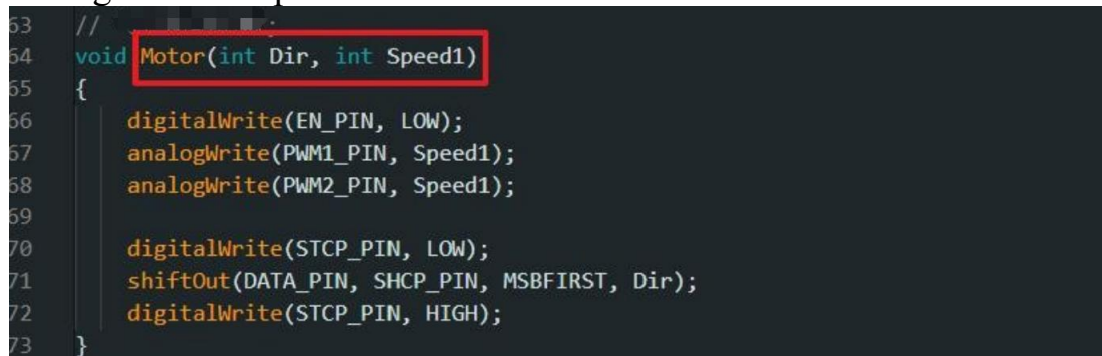
    delay(500);

}

}

}
```

Program Description:



```
53 //
54 void Motor(int Dir, int Speed1)
55 {
56     digitalWrite(EN_PIN, LOW);
57     analogWrite(PWM1_PIN, Speed1);
58     analogWrite(PWM2_PIN, Speed1);
59
60     digitalWrite(STCP_PIN, LOW);
61     shiftOut(DATA_PIN, SHCP_PIN, MSBFIRST, Dir);
62     digitalWrite(STCP_PIN, HIGH);
63 }
```

Speed1 is the speed variable, which is used to adjust the speed in the program. When calling the function, pass in a value of 100~250 to control the car to execute different speeds.

```

119     else if('B' == cmd)
120     {
121         Motor(Backward,250);
122     }
123     else if('L' == cmd)
124     {
125         Motor(Left,250);
126     }
127     else if('R' == cmd)
128     {
129         Motor(Right,250);

```

4.2. WiFi Remote Control 2_ESP32.ino

Before burning the program, change the WiFi account name and password. As shown below, change "XXX" to your own WiFi name and "XXXXXX" to the password.

```
const char* ssid = "XXX"; //Enter SSID WIFI Name Enter "SSID WIFI
Name"
```

```
const char* password = "XXXXXX"; //Enter WIFI Password Enter WIFI
password
```

More code details :

```
#include "esp_camera.h"
```

```
#include "esp_timer.h"
```

```
#include "img_converters.h"
```

```
#include "Arduino.h"
```

```
#include "fb_gfx.h"
```

```
#include "soc/soc.h" //disable brownout problems
```

```
#include "soc/rtc_cntl_reg.h" //disable brownout problems
```

```
#include "esp_http_server.h"

#include <WiFi.h>

//

// WARNING!!! Make sure that you have either selected ESP32 Wrover
Module,

// or another board which has PSRAM enabled

//

// Adafruit ESP32 Feather

// Select camera model

// #define CAMERA_MODEL_WROVER_KIT

// #define CAMERA_MODEL_M5STACK_PSRAM

#define CAMERA_MODEL_AI_THINKER

const char* ssid = "xxx"; //Enter SSID WIFI Name Enter "SSID WIFI
Name"

const char* password = "xxxxxx"; //Enter WIFI Password Enter WIFI
password

#if defined(CAMERA_MODEL_WROVER_KIT)

#define PWDN_GPIO_NUM -1
```

```
#define RESET_GPIO_NUM -1
```

```
#define XCLK_GPIO_NUM 21
```

```
#define SIOD_GPIO_NUM 26
```

```
#define SIOC_GPIO_NUM 27
```

```
#define Y9_GPIO_NUM 35
```

```
#define Y8_GPIO_NUM 34
```

```
#define Y7_GPIO_NUM 39
```

```
#define Y6_GPIO_NUM 36
```

```
#define Y5_GPIO_NUM 19
```

```
#define Y4_GPIO_NUM 18
```

```
#define Y3_GPIO_NUM 5
```

```
#define Y2_GPIO_NUM 4
```

```
#define VSYNC_GPIO_NUM 25
```

```
#define HREF_GPIO_NUM 23
```

```
#define PCLK_GPIO_NUM 22
```

```
#elif defined(CAMERA_MODEL_AI_THINKER)
```

```
#define PWDN_GPIO_NUM 32
```

```
#define RESET_GPIO_NUM -1
```

```
#define XCLK_GPIO_NUM 0
```

```
#define SIOD_GPIO_NUM 26
```

```
#define SIOC_GPIO_NUM 27
```

```
#define Y9_GPIO_NUM 35
```

```
#define Y8_GPIO_NUM 34
```

```
#define Y7_GPIO_NUM 39
```

```
#define Y6_GPIO_NUM 36
```

```
#define Y5_GPIO_NUM 21
```

```
#define Y4_GPIO_NUM 19
```

```
#define Y3_GPIO_NUM 18
```

```
#define Y2_GPIO_NUM 5
```

```
#define VSYNC_GPIO_NUM 25
```

```
#define HREF_GPIO_NUM 23
```

```
#define PCLK_GPIO_NUM 22
```

```
#else
```

```
#error "Camera model not selected"
```

```
#endif
```

```
extern int gpLed = 4; // Light
```

```
extern String WiFiAddr = "";
```

```
void startCameraServer();

void setup() {
  //Serial.begin(115200);

  Serial.begin(9600); // Match the Arduino speed that controls the motors

  // Serial.setDebugOutput(true);

  Serial.println();

  /*

  pinMode(gpLb, OUTPUT); //Left Backward
  pinMode(gpLf, OUTPUT); //Left Forward
  pinMode(gpRb, OUTPUT); //Right Forward
  pinMode(gpRf, OUTPUT); //Right Backward

  //initialize

  digitalWrite(gpLb, LOW);
  digitalWrite(gpLf, LOW);
  digitalWrite(gpRb, LOW);
  digitalWrite(gpRf, LOW);

  */

  pinMode(gpLed, OUTPUT); //Light
```

```
digitalWrite(gpLed, LOW);
```

```
camera_config_t config;
```

```
config.ledc_channel = LEDC_CHANNEL_0;
```

```
config.ledc_timer = LEDC_TIMER_0;
```

```
config.pin_d0 = Y2_GPIO_NUM;
```

```
config.pin_d1 = Y3_GPIO_NUM;
```

```
config.pin_d2 = Y4_GPIO_NUM;
```

```
config.pin_d3 = Y5_GPIO_NUM;
```

```
config.pin_d4 = Y6_GPIO_NUM;
```

```
config.pin_d5 = Y7_GPIO_NUM;
```

```
config.pin_d6 = Y8_GPIO_NUM;
```

```
config.pin_d7 = Y9_GPIO_NUM;
```

```
config.pin_xclk = XCLK_GPIO_NUM;
```

```
config.pin_pclk = PCLK_GPIO_NUM;
```

```
config.pin_vsync = VSYNC_GPIO_NUM;
```

```
config.pin_href = HREF_GPIO_NUM;
```

```
config.pin_sscb_sda = SIOD_GPIO_NUM;
```

```
config.pin_sscb_scl = SIOC_GPIO_NUM;
```

```
config.pin_pwdn = PWDN_GPIO_NUM;
```

```
config.pin_reset = RESET_GPIO_NUM;
```

```
config.xclk_freq_hz = 20000000;
```

```
config.pixel_format = PIXFORMAT_JPEG;

//init with high specs to pre-allocate larger buffers

if(psramFound()){

config.frame_size = FRAMESIZE_UXGA;

config.jpeg_quality = 10;

config.fb_count = 2;

} else {

config.frame_size = FRAMESIZE_SVGA;

config.jpeg_quality = 12;

config.fb_count = 1;

}

// camera init

esp_err_t err = esp_camera_init(&config);

if (err != ESP_OK) {

// Serial.printf("Camera init failed with error 0x%x", err); printf("Camera

initialization failed, error 0x%x", err);

return;

}

//drop down frame size for higher initial frame rate

sensor_t * s = esp_camera_sensor_get();

s->set_framesize(s, FRAMESIZE_CIF);
```



```
WiFi.begin(ssid, password);

while (WiFi.status() != WL_CONNECTED)
{
  delay(500);
  Serial.print(".");
}

Serial.println("");
Serial.println("WiFi connected");
startCameraServer();
Serial.print("Camera Ready! Use 'http://");
Serial.print(WiFi.localIP());
WiFiAddr = WiFi.localIP().toString();
Serial.println(" to connect");
}

void loop()
{
  // put your main code here, to run repeatedly:
}
```

Code Explanation :

In addition to the main program code 2_ESP32.ino, this project code also includes C++ library code, which may be used when some custom

settings are needed. For example, to change the camera screen angle:

Click `app_httpd.cpp` next to the main program, press `ctrl+F` on the keyboard to search and modify the `deg` parameter, so that you can set the camera perspective at different angles.



```
285
286     esp_err_t index_handler(httpd_req_t *req){
287         httpd_resp_set_type(req, "text/html");
288         string page = "";
289         += "<meta name=\"viewport\" content=\"width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=0\">\n";
290         += "<script>var xhttp = new XMLHttpRequest();</script>";
291         += "<script>function getsend(arg) { xhttp.open('GET', arg +'?' + new Date().getTime(), true); xhttp.send() } </script>";
292         += "<p align=center><IMG SRC='http://\" + WiFiAddr + \":81/stream' style='width:300px; transform:rotate(180deg);'></p><br/>";
293
294
295         += "<div";
```

-90/90/180

Of course, there are some window sizes that can also be set by modifying the "width" parameter value.